ME 325 — Engineering Design Project | 2026

TwinTalk

Voice-Actuated Tracking System for the Twin Rotor Platform

Project Proposal Report

E/21/338 **Riswan**	E/21/205 **Shathurshiga**	E/21/100 **Dilshan**

Department of Mechanical Engineering
University of Peradeniya

Submitted: May 2026

1. Executive Summary

TwinTalk is a modular, voice-actuated control system for the Twin Rotor V1.0 aerodynamic laboratory platform developed by Orise for the Department of Mechanical Engineering, University of Peradeniya. The project addresses a critical gap in current laboratory practice: operators must interact with rotor platforms through keyboard-based or tethered interfaces, imposing physical constraints and cognitive overhead that are ill-suited to dynamic experimental scenarios.

TwinTalk enables hands-free trajectory control through natural spoken language. A Flutter/Dart mobile application captures the operator's voice command and forwards it to an AI orchestrator built on LangGraph. The orchestrator parses intent, optionally runs a MeshCat kinematic simulation for safety preview, requests voice authorization from the operator (Human-in-the-Loop, HiL), and — upon approval — dispatches precise angular trajectory commands to a Raspberry Pi controller. The controller executes PID and Geometric control algorithms to generate PWM signals for the twin rotor motors.

The project is structured across four phases spanning 14 weeks, culminating in a functional pipeline, advanced Geometric control implementation, and a final technical report. All software is developed on an open-source Python stack with a Dart/Flutter mobile frontend, making results fully reproducible and directly transferable to the commercial UAV research community.

2. Introduction and Motivation

2.1 Background

The Twin Rotor V1.0 is a two-degree-of-freedom (2-DOF) benchtop aerodynamic platform featuring two independently driven propeller-motor assemblies on a shared pivot structure. Its onboard electronics include a 9-axis IMU, a yaw encoder, a slip ring for continuous rotation, and an STM32 microcontroller that communicates with a Raspberry Pi over a CAN bus. Power is supplied at 230V AC, stepped down to 16V DC for the motors and 5V for logic.

In its current deployment, the platform is controlled through a desktop GUI or direct serial commands, requiring the operator to remain at a workstation. This constraint is problematic in several experimental contexts: vibration testing, where the operator must stand clear; multi-platform scenarios, where one operator manages several rigs simultaneously; and accessibility contexts, where motor impairments may prevent keyboard operation.

2.2 Problem Statement

No existing control interface for the Twin Rotor V1.0 supports hands-free, natural language operation. Furthermore, voice-controlled robotic platforms in the literature typically lack a pre-execution safety simulation layer, creating risk of hardware damage or operator injury when AI-generated commands are executed without verification. TwinTalk addresses both gaps.

2.3 Project Scope

This proposal defines the design, implementation, and validation of a voice-actuated control pipeline for the Twin Rotor V1.0. The system scope encompasses:

- A Flutter/Dart mobile application with integrated speech capture and voice authorization
- An AI orchestration layer using LangGraph for multi-agent intent parsing and command generation
- A MeshCat-based 3D kinematic simulation for pre-execution trajectory preview
- A Raspberry Pi embedded controller implementing PID and Geometric control algorithms
- End-to-end Bluetooth/Wi-Fi communication between all system components

3. Project Objectives

3.1 Primary Objective

To develop and validate a modular, voice-actuated control system (TwinTalk) for the Twin Rotor V1.0 that enables hands-free, precise angular trajectory tracking through natural language commands, with mandatory human-in-the-loop safety authorization before any physical hardware motion.

3.2 Specific Objectives

1. Design and implement a Flutter/Dart mobile application that captures voice commands, forwards them to the AI orchestrator, receives simulation previews, and allows voice-based motion authorization.
2. Develop a LangGraph multi-agent orchestration system that transcribes speech using OpenAI Whisper, parses operator intent, extracts trajectory parameters (axis, direction, magnitude), and generates structured motor commands.
3. Build a MeshCat-based 3D kinematic simulation environment that renders the projected Twin Rotor arm trajectory in real time, enabling visual verification before hardware execution.
4. Implement a linearized PID controller on the Raspberry Pi for initial integration and a Geometric controller (SO(3)-based) for robust large-angle trajectory tracking in the final project phase.
5. Validate the complete TwinTalk pipeline through step-response testing, trajectory tracking trials, and user acceptance evaluation in the Mechanical Engineering laboratory.

4. System Architecture

4.1 Architecture Overview

TwinTalk follows a Human-in-the-Loop (HiL) architecture where voice commands flow through a chain of processing layers before hardware actuation is permitted. The architecture is intentionally layered so that each stage can be developed, tested, and validated independently. Figure 1 (described textually below) summarises the six-step pipeline.

TwinTalk Pipeline — Six-Step Flow

Step 1 → Operator speaks command into Flutter/Dart mobile app

Step 2 → App transmits audio/text to laptop AI orchestrator via Bluetooth / Wi-Fi

Step 3 → LangGraph agent decides if simulation is required

Step 4 → If required: MeshCat renders 3D kinematic preview; results sent to app

Step 5 → Operator gives voice authorization via mobile app (APPROVE / REJECT)

Step 6 → Laptop sends approved command to Raspberry Pi → motors execute trajectory

4.2 Layer 1: Mobile Interface (Flutter/Dart)

The mobile application is developed using Flutter and the Dart programming language, targeting Android devices. Flutter's widget-based reactive framework enables a responsive UI with low latency audio capture. The application provides three primary screens: (1) a voice command screen with a hold-to-record button that streams audio to the AI orchestrator; (2) a simulation preview screen that displays the MeshCat trajectory result returned by the orchestrator; and (3) an authorization screen that presents the parsed command in natural language and records the operator's voice approval or rejection.

Dart's asynchronous programming model (async/await with Futures and Streams) is well-suited to the real-time audio capture and WebSocket communication required by TwinTalk. The app communicates with the laptop orchestrator over Bluetooth for short-range lab use and Wi-Fi for higher-throughput simulation preview streaming.

4.3 Layer 2: AI Orchestrator (LangGraph + OpenAI Whisper)

The orchestration layer runs on a laptop as a Python service. It receives raw audio from the mobile app, passes it to OpenAI Whisper for transcription, then feeds the transcript into a LangGraph stateful agent graph. The agent graph contains the following nodes:

- Transcription Node: Calls Whisper API with the audio buffer; returns plain-text transcript.
- Intent Parser Node: Uses an LLM with structured output prompting to extract a JSON command object containing axis (pitch/yaw), direction (positive/negative), and magnitude (degrees).
- Feasibility Check Node: Validates the extracted command against the Twin Rotor's physical limits (e.g. pitch ±90°, yaw continuous).
- Simulation Router Node: Decides — based on command magnitude and current platform state — whether a MeshCat simulation is required before proceeding.
- Authorization Node: Waits for the HiL approval signal from the mobile app before dispatching the command.
- Dispatch Node: Sends the validated, authorized command to the Raspberry Pi via socket.

4.4 Layer 3: Simulation Safety Gate (MeshCat)

MeshCat is a web-based, Three.js-powered 3D visualizer that is part of the Drake robotics toolkit. TwinTalk uses MeshCat to render a kinematic model of the Twin Rotor V1.0 arm, animating the proposed trajectory before execution. The simulation runs forward kinematics using the platform's known geometry (arm span 732.40 mm, propeller radius 115 mm, pole height 583.28 mm as documented in the hardware manual) to produce a realistic preview of the planned motion.

The simulation output — a short animated preview and a pass/fail feasibility flag — is streamed back to the Flutter app for operator review. This gate cannot be bypassed: the Dispatch Node in LangGraph will not emit a command unless the authorization signal has been received from the mobile app following the simulation preview.

4.5 Layer 4: Embedded Controller (Raspberry Pi)

The Raspberry Pi runs a Python-based real-time control loop that receives trajectory waypoints from the laptop orchestrator. In Phase 3 (Weeks 8-9), a linearized PID controller is implemented to provide initial closed-loop control using feedback from the onboard yaw encoder and 9-axis IMU. In Phase 4 (Weeks 10-14), this is upgraded to a Geometric controller operating on the Special Orthogonal Group SO(3), which avoids Euler angle singularities and maintains stability during large-angle maneuvers.

The Raspberry Pi interfaces with the STM32 MCU via the existing CAN bus (CAN controller + transceiver as shown in the hardware electrical diagram). The STM32 translates CAN frame commands into PWM signals for Motor 1 and Motor 2. The Raspberry Pi also reads IMU and encoder data via the slip ring, closing the feedback loop at the embedded control layer.

5. Technical Approach

5.1 Speech Recognition Pipeline

Voice commands are captured at 16 kHz mono PCM using the Flutter sound plugin on Android. The audio buffer is compressed and transmitted to the laptop orchestrator. OpenAI Whisper (medium.en model) transcribes the audio with word-error rates below 5% under typical laboratory conditions. The Whisper API call returns a JSON response containing the transcript text, which is immediately passed to the LangGraph intent parser.

To improve robustness under motor noise and fan interference, a voice activity detection (VAD) pre-filter is applied to the audio buffer before transmission, reducing false activations. The operator is notified of transcription confidence via a visual indicator on the Flutter UI.

5.2 Command Parsing and Structured Output

The LangGraph intent parser uses a system prompt that constrains the LLM to respond only with a valid JSON object matching a fixed schema: {"axis": "pitch|yaw", "direction": "positive|negative", "magnitude_deg": <float>, "trajectory_type": "step|ramp|sinusoid"}. This structured output approach eliminates free-text ambiguity and allows the Feasibility Check Node to perform deterministic limit validation. Commands that fail validation are rejected with a natural language explanation returned to the mobile app.

5.3 Kinematic Simulation

The MeshCat simulation models the Twin Rotor V1.0 as a two-link kinematic chain with a fixed base. Link 1 represents the vertical pole (height 583.28 mm); Link 2 represents the horizontal arm (half-span 366.2 mm); propeller discs are rendered at each arm end (radius 115 mm). Joint 1 is the yaw axis (continuous rotation); Joint 2 is the pitch axis (±90° limit). Forward kinematics are computed using homogeneous transformation matrices and animated at 30 frames per second.

The simulation detects potential issues including: joint limit violation, propeller-ground clearance breach (minimum safe height from the hardware manual), and excessive angular velocity that could cause structural stress. Any detected issue triggers an automatic REJECT flag, and the operator is notified before the authorization step.

5.4 Control Algorithms

5.4.1 Linearized PID Controller (Phase 3)

The PID controller is implemented as a discrete-time algorithm running at 100 Hz on the Raspberry Pi. Separate PID loops govern pitch and yaw axes. Gains are tuned using the Ziegler-Nichols step-response method on the physical platform. Anti-windup clamping is applied to the integral term to prevent actuator saturation during large reference steps. The control output is a PWM duty cycle percentage mapped to the motor thrust range.

5.4.2 Geometric Controller on SO(3) (Phase 4)

The Geometric controller follows the formulation of Lee et al. [5], defining attitude error on the Lie algebra so(3) as eR = (1/2)(Rd^T R − R^T Rd)^{∨}, where R is the current rotation matrix, Rd is the desired rotation matrix, and ^∨ denotes the vee map. The control torque is τ = −kR eR − kΩ eΩ + Ω × JΩ, where eR and eΩ are attitude and angular velocity errors, J is the inertia matrix, and kR, kΩ are positive control gains. This formulation guarantees almost-global exponential stability on SO(3).

6. Technology Stack

Component	Technology / Tool
Mobile Application	Flutter / Dart (Android) — voice capture, HiL authorization, simulation preview UI
Speech Recognition	OpenAI Whisper API (medium.en model) — cloud-side ASR
AI Orchestration	LangGraph (Python) — stateful multi-agent pipeline with conditional routing
Language Model	OpenAI GPT-4o — intent parsing and structured JSON command generation
3D Simulation	MeshCat (Python/Three.js) — real-time kinematic preview and safety gate
Communication	Bluetooth / Wi-Fi — mobile-to-laptop; TCP socket — laptop-to-Raspberry Pi
Embedded Control	Raspberry Pi OS (Python) — PID and Geometric control loop
MCU Interface	STM32 over CAN bus — PWM signal generation for Motor 1 and Motor 2
Sensor Fusion	9-Axis IMU + Yaw Encoder (onboard Twin Rotor V1.0)
Version Control	Git / GitHub — open-source repository
Simulation Math	NumPy, SciPy — kinematics and control algorithm implementation
Hardware Control Sim	MATLAB/Simulink — control algorithm verification before deployment

7. Project Plan and Milestones

7.1 Project Timeline

The project is organized across four phases over 14 weeks. Each phase concludes with a defined milestone deliverable.

Phase & Timeline	Key Tasks
Phase 1: Project Kickoff Weeks 1–3	• Literature review and problem formulation • GitHub repository setup and branching strategy • Basic conceptualization and interface wireframing
Phase 2: Detailed Design Weeks 4–7	• Lang Graph agent routing and tool definitions • Mesh Cat simulation environment setup • Mobile app skeleton (Dart / Flutter) • Wireframing of HiL authorization flow
Phase 3: Initial Integration Weeks 8–9	• Connect full end-to-end pipeline • Test system functionality • Implement linearized PID controller on Raspberry Pi
Phase 4: Geometric Upgrade Weeks 10–14	• Implement advanced Geometric control algorithms (SO(3)) • Fine-tune physical robustness and noise handling • Finalize technical report and documentation

7.2 Milestone Deliverables

Milestone	Deliverable
End of Week 3	Kickoff complete: literature review, repository, conceptual design
End of Week 7	Design complete: Lang Graph routing, Mesh Cat environment, app skeleton, wireframes
End of Week 9	Integration complete: functional end-to-end pipeline with PID controller
End of Week 14	Project complete: Geometric control, tuned system, final report and demonstration

8. Risk Analysis and Mitigation

Risk	Likelihood	Mitigation Strategy
High ASR error rate in noisy lab environment	**Medium**	VAD pre-filtering; Whisper medium model; fallback manual text input on Flutter UI
LangGraph intent parsing returns invalid command	**Low**	Structured JSON output schema with validation; feasibility check node rejects malformed commands
MeshCat simulation latency exceeds operator tolerance	**Medium**	Async simulation with streaming preview; operator can bypass simulation for pre-validated commands
CAN bus communication failure during execution	**Low**	Watchdog timer on STM32; safe-stop PWM signal sent on timeout; hardware yaw axis lock as emergency stop
Geometric controller instability on physical hardware	**Medium**	Extensive MATLAB/Simulink verification before deployment; PID fallback always available
Twin Rotor hardware damage during testing	**Low**	MeshCat safety gate enforces joint limits; propeller guards fitted; testing begins at low PWM duty cycles
Bluetooth range or pairing issues in lab	**Low**	Wi-Fi fallback; both protocols implemented in Flutter/Dart communication layer

9. Expected Outcomes and Deliverables

9.1 Technical Deliverables

- A fully functional Flutter/Dart Android application for voice-command capture and HiL authorization
- A LangGraph multi-agent orchestration service with Whisper ASR and structured command parsing
- A MeshCat kinematic simulation environment pre-configured with the Twin Rotor V1.0 geometry
- A Raspberry Pi control module implementing both PID and Geometric (SO(3)) controllers
- An open-source GitHub repository containing all source code, documentation, and test scripts
- A final technical report documenting system design, implementation, experimental results, and conclusions

9.2 Performance Targets

Metric	Target
Voice command end-to-end latency (capture to dispatch)	< 3 seconds (without simulation)
ASR word error rate (lab conditions)	< 8%
Angular tracking error (PID, step input)	< 5° steady-state
Angular tracking error (Geometric, step input)	< 2° steady-state
HiL authorization round-trip time	< 5 seconds
MeshCat simulation render time	< 2 seconds for full trajectory preview
System uptime per session	> 95% (no crash requiring restart)

9.3 Impact Analysis

Economic Impact

TwinTalk is built entirely on an open-source Python stack (LangGraph, MeshCat, NumPy, SciPy) combined with a Dart/Flutter mobile frontend — eliminating software licensing fees. The system demonstrates clear ROI for AI-augmented laboratory infrastructure and provides engineering skills directly transferable to the commercial UAV and aerospace robotics industry.

Social Impact

Hands-free operation significantly reduces operator cognitive load in complex experimental scenarios. The intuitive mobile interface and voice command capabilities lower the barrier to entry for rotor platform operation, making the Twin Rotor accessible to researchers and students who may have motor impairments.

Environmental Impact

By testing trajectories in the MeshCat digital simulator before physical execution, TwinTalk reduces the number of hardware test runs required, decreasing motor wear and energy consumption over the project lifetime. Efficient AI inference code minimises computational power draw on both the mobile device and the laptop orchestrator.

10. Literature Review Summary

A full literature review accompanies this proposal as a separate document. This section summarises the key findings relevant to design decisions.

10.1 Twin Rotor Control

Ahmad et al. [1] established the coupled nonlinear dynamics of twin-rotor MIMO systems, motivating the use of compensation terms in the PID controller. Lee et al. [5] provided the SO(3)-based Geometric controller formulation adopted in TwinTalk's Phase 4 upgrade, guaranteeing almost-global exponential stability without Euler angle singularities.

10.2 Speech Recognition

Radford et al. [9] demonstrated that OpenAI Whisper achieves near-human word-error rates on English speech with robust performance under ambient noise. Tellex et al. [10] and Ahn et al. [11] established NLU grounding frameworks for translating natural language to robot actions, informing TwinTalk's LangGraph intent parser design.

10.3 AI Agent Orchestration

Yao et al. [12] introduced ReAct-style reasoning-action agents, the conceptual basis for TwinTalk's orchestrator. Chase et al. [13] developed LangGraph for stateful, cyclic agent pipelines, directly enabling TwinTalk's conditional simulation-routing architecture.

10.4 Human-in-the-Loop Safety

Sheridan and Verplank [14] established the supervisory control taxonomy that frames TwinTalk's HiL design. Siciliano et al. [15] documented pre-execution simulation as standard industrial practice, validating TwinTalk's MeshCat safety gate approach.

10.5 Flutter/Dart for Mobile Robotics Interfaces

Flutter's widget framework and Dart's async programming model provide low-latency audio capture and responsive UI updates suitable for real-time robot control interfaces. The platform's cross-compilation capability and strong typing make it preferable to React Native for safety-critical mobile applications where runtime type errors must be caught at compile time.

11. Conclusion

TwinTalk proposes a technically rigorous, practically motivated voice-actuated control system for the Twin Rotor V1.0 laboratory platform. By integrating Flutter/Dart mobile development, LangGraph AI orchestration, OpenAI Whisper speech recognition, MeshCat kinematic simulation, and Geometric control theory, the project delivers a comprehensive bridge between natural human speech and precision aerodynamic hardware.

The project is structured to deliver meaningful results at each milestone: a working end-to-end pipeline by Week 9 and a fully optimised, Geometric-controlled system by Week 14. All software will be released as open-source, and the methodology is directly transferable to other multi-rotor laboratory platforms and commercial UAV control research.

The team — Riswan (E/21/338), Shathurshiga (E/21/205), and Dilshan (E/21/100) — collectively brings expertise in AI systems, mobile application development, and embedded control, providing comprehensive coverage across TwinTalk's technology stack. This proposal requests approval to proceed to Phase 1 implementation.

References

[1] S. Ahmad, A. Chipperfield, and M. Tokhi, "Dynamic modelling and linear quadratic Gaussian control of a twin-rotor multi-input multi-output system," Proc. Inst. Mech. Eng. I, vol. 217, no. 3, pp. 203-227, 2003.

[2] T. Bresciani, "Modelling, identification and control of a quadrotor helicopter," M.S. thesis, Dept. Automatic Control, Lund Univ., Lund, Sweden, 2008.

[3] F. L. Santoso et al., "Multi-model PID controller design for a twin rotor MIMO system," in Proc. IEEE Conf. Control Appl., 2010, pp. 1404-1409.

[4] Orise, "Twin Rotor V1.0 User Manual," Dept. Mechanical Engineering, University of Peradeniya, Oct. 2023.

[5] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in Proc. 49th IEEE Conf. Decision and Control, 2010, pp. 5420-5425.

[6] F. Bullo and R. M. Murray, "Proportional derivative (PD) control on the Euclidean group," in Proc. 3rd Eur. Control Conf., 1995.

[7] A. Graves et al., "Connectionist temporal classification," in Proc. 23rd ICML, 2006, pp. 369-376.

[8] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[9] A. Radford et al., "Robust speech recognition via large-scale weak supervision," in Proc. 40th ICML, 2023, pp. 28492-28518.

[10] S. Tellex et al., "Understanding natural language commands for robotic navigation," in Proc. 25th AAAI, 2011, pp. 1507-1514.

[11] M. Ahn et al., "Do as I can, not as I say: Grounding language in robotic affordances," in Proc. 6th CoRL, 2022, pp. 287-318.

[12] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. 11th ICLR, 2023.

[13] H. Chase et al., "LangGraph: Building stateful, multi-actor applications with LLMs," LangChain, 2024. [Online]. Available: https://github.com/langchain-ai/langgraph

[14] T. B. Sheridan and W. L. Verplank, "Human and computer control of undersea teleoperators," MIT Man-Machine Systems Lab, Tech. Rep., 1978.

[15] B. Siciliano et al., Robotics: Modelling, Planning and Control. London, U.K.: Springer, 2009.

[16] R. Tedrake and the Drake Development Team, "Drake: Model-based design and verification for robotics," 2019. [Online]. Available: https://drake.mit.edu

[17] D. Molloy, Exploring Raspberry Pi: Interfacing to the Real World with Embedded Linux. Hoboken, NJ: Wiley, 2016.

[18] R. Bosch GmbH, "CAN Specification Version 2.0," Bosch, Stuttgart, Germany, Tech. Rep., 1991.

[19] A. Hughes and B. Drury, Electric Motors and Drives, 4th ed. Oxford, U.K.: Newnes, 2013.